

应用层的作用是为用户的分布式应用程序提供访问网络的接口。

2.1 应用层的基本概念

2.1.1 网络应用程序的体系结构

网络应用程序的体系结构指在**各端系统上的分布式应用程序的工作模式**，由研发者决定，分为客户机/服务器模式(C/S)和对等模式(P2P)。

客户机/服务器模式：服务器通常一直运行，具有固定的IP地址，可为**多个客户机**提供服务，可扩展为服务器集群。客户机偶尔开机，具有固定或动态的IP地址，向服务器发出请求，客户机之间**不直接通信**。大部分网络应用如Web应用、电子邮件、文件传输采用该模式。

P2P模式：**无(或很少使用)服务器**。如果有服务器，也只用于如注册IP地址、登记用户、计费等。端系统(对等方)之间**直接数据通信**，既请求服务，也提供服务。对等方**IP地址可以不固定**。BT、QQ、Skype等采用该模式。

2.1.2 进程通信

进程(process)即在主机上并行运行的程序。同一主机中的进程通信由**操作系统**控制，不同主机中的进程通信通过**网络**交换报文，发送进程产生报文，并向网络发送；接收进程接收报文，并返回响应报文。

分布式应用程序由**不同进程组成**，发起通信的进程为**客户机(client)进程**，等待通信的进程为**服务器(server)进程**。在P2P模式中，一个进程同时充当两角色。

网络通信实质上是不同主机进程间通信，网络中传送的分组，通常携带目的进程的识别信息。进程的识别信息分两部分。**主机地址指出进程位于网络中的哪一个主机**，因特网中用**IP地址**标识。**进程标识指出进程是主机中的哪一个进程**，在主机中用**传输层协议和端口号**标识。

端口用来在主机中区分不同的进程，针对TCP和UDP，各有一套0~65535的端口。端口分为周知端口和临时端口。周知端口有固定用途，通常用于特定的服务器进程，编号0~1023，如Web服务器进程TCP 80，邮件传输服务器进程TCP 25，DHCP服务器进程UDP 67。临时端口一般用于客户机进程，使用前由进程向本地操作系统申请，使用完毕后归还，编号1024~65535。

2.1.3 套接字

套接字(socket)：位于应用层的应用程序(进程)和网络之间的**应用程序接口API**，位于**应用层与运输层**之间，用于进程向网络发送和接收报文。进程与套接字好比房子与门，每个进程通过对应的套接字在网络上收发报文。编程者设计套接字的应用层端，即进程，几乎不用控制套接字的运输层端，但可选择运输层协议，设定运输层参数。

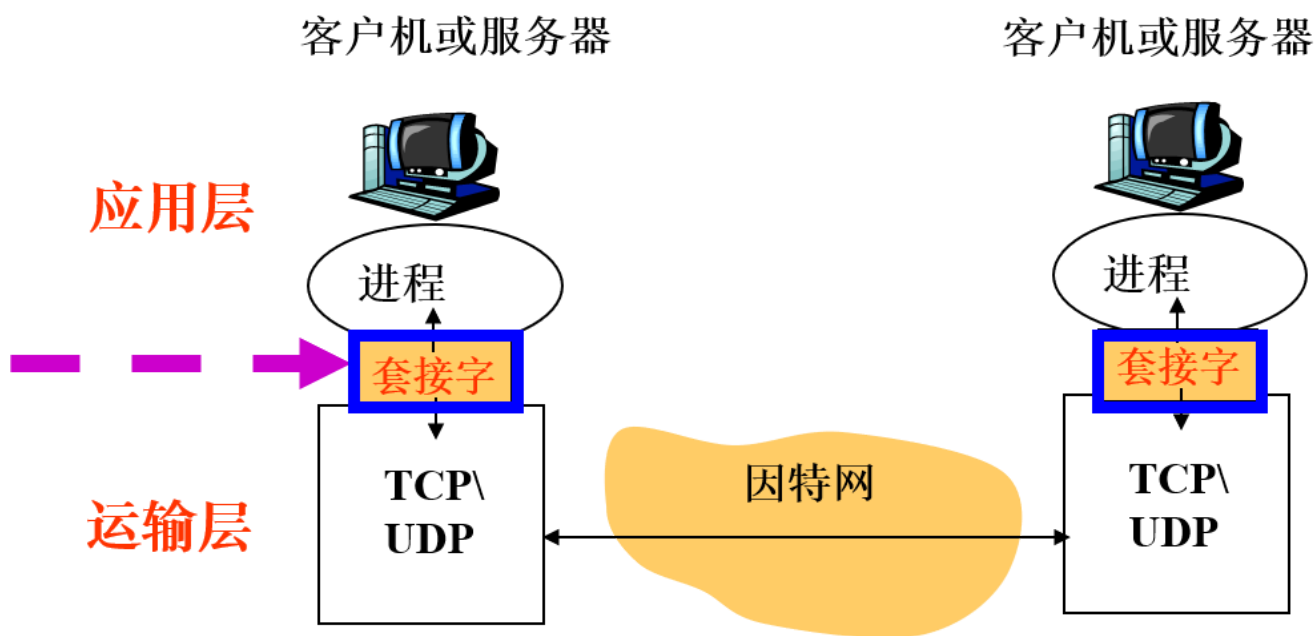


图2-1：套接字

2.1.4 应用层协议

应用层协议**定义了网络进程间传递报文的格式和规则**。具体有**语法**：各类报文的各字段结构；**语义**：字段信息的含义；**语序**：何时、如何发送报文及对报文进行响应。

网络应用所需的运输服务有可靠数据传输、吞吐量、定时和安全性。

应用程序	数据丢失	带宽	时间敏感
文件传输	不能丢失	弹性	否
电子邮件	不能丢失	弹性	否
Web文档	不能丢失	弹性	否
实时音/视频	容忍丢失	音频：5kbps~1Mbps 视频10kbps~5Mbps	是，100ms
存储音/视频	容忍丢失	同上	是，几秒
交互式游戏	容忍丢失	几kbps以上	是，100ms
即时讯息	不能丢失	弹性	是

应用层协议建立在运输层提供的服务之上。**因特网的两个运输层协议TCP和UDP提供不同的进程到进程传输服务。**

TCP服务是面向连接、可靠的传输服务。客户机程序和服务器程序之间交换控制信息，在两个进程的**套接字之间通过网络建立一个TCP连接**，通过**全双工方式**进行报文传输，最后拆除连接。TCP采取确认重传、流量控制、拥塞控制等措施实现进程间**无差错、按顺序**的数据交付。TCP能**保证正确交付**所有的数据，但开销较大，**适合重要数据、较大数据的可靠传输**。

UDP服务采用**最小服务模式**，进程间无握手过程，不保证报文能够被目的进程最终接收和按序接收，以当前最大速率传输，没有流量控制、拥塞控制机制，也不提供时延保证。**UDP简单高效，适于普通数据、小块数据的快速传送。**

应用	应用层协议	传输层协议
电子邮件	SMTP	TCP
远程终端访问	Telnet	TCP
Web	HTTP	TCP
文件传输	FTP	TCP
远程文件服务器	NFS	UDP或TCP
流媒体	HTTP、RTP	UDP或TCP
因特网电话	SIP、RTP	UDP或TCP

2.2 Web应用

2.2.1 HTTP概述

Web应用包括客户及程序(浏览器)和服务程序，运行在不同的端系统中，通过应用层的**HTTP(Hypertext Transfer Protocol超文本传输协议)**进行会话。HTTP协议定义了**HTTP报文的格式**以及客户机和服务器**交换报文的方式**。

Web页面(文档)也称网页，由若干**对象文件**组成，如HTML(Hypertext Markup Language 超文本标记语言)文件、JPEG图形、Java小程序等，**每个对象由URL来寻址**。

URL(Uniform Resource Locator)：统一资源定位符，用于标识万维网上对象资源的全网唯一的地址。

客户机程序(浏览器)：用于向服务器程序发出Web页面获取请求，并向用户显示所获取的Web页面。如IE，FireFox，Chrome等。

Web服务器程序：用于存贮Web对象(由URL寻址)，并响应客户请求。如IIS(Windows，商业软件)，Apache(跨平台，开放软件，应用最多)，TomCat(跨平台，开放软件，面向Java)。

Web应用使用**客户机/服务器**模式，服务器**总是打开**，使用**固定IP地址和周知端口80**，为**多个浏览器**服务。用户点击或输入超链接，浏览器向URL对应的服务器**发出HTTP请求报文**，服务器接收请求，返回包含页面(若存在)的**HTTP响应报文**。

HTTP基于运输层的TCP，保证HTTP报文传输的**可靠性**。HTTP是**无状态协议**，服务器不保存关于客户机的任何信息。

2.2.2 HTTP连接

HTTP1.1默认使用持续连接方式，一个TCP连接上可以传送多对HTTP请求/响应报文。

持续连接方式有流水线方式(默认)和非流水线方式两种。在同一TCP连接上，非流水线方式中客户机只能在**收到前一个请求的响应之后，才能发出新的请求**。流水线方式中客户机**连续发送请求**(针对不同的Web页面)，无需等待响应。服务器**连续返回此客户机请求的对象**。

2.2.3 HTTP报文格式

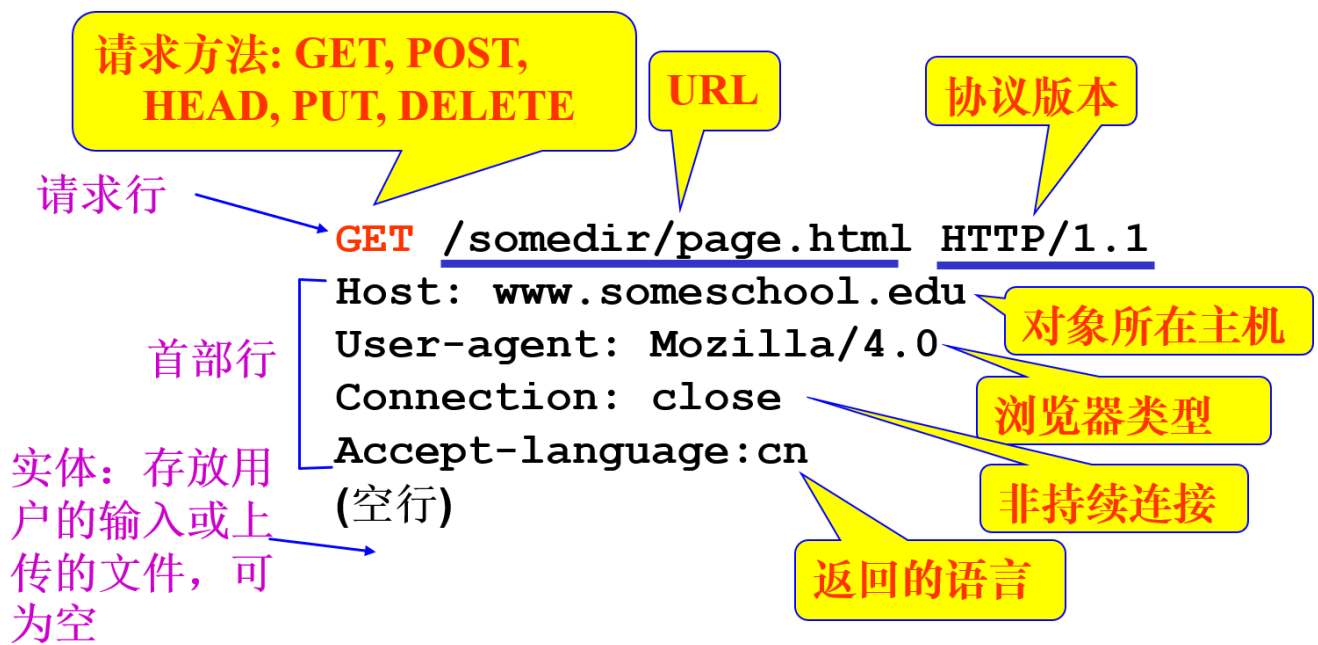


图2-2: HTTP请求报文的格式

请求方法:

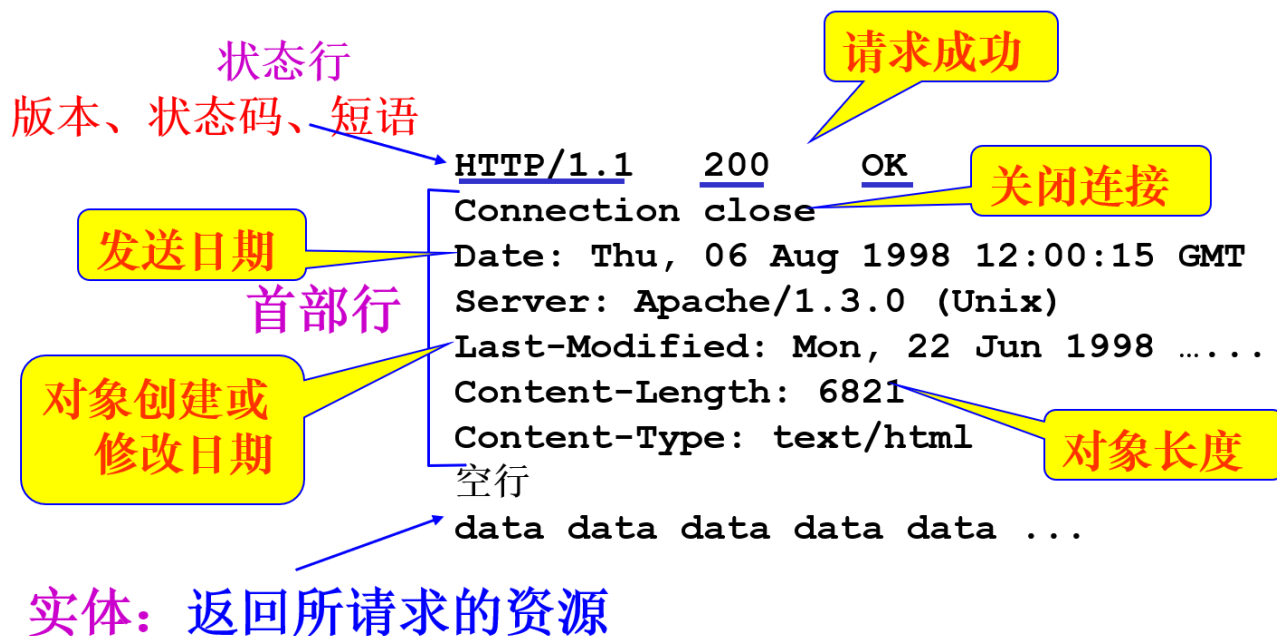
GET: 请求一个对象, 如Web页面, 此时实体为空。

POST: 将实体中信息传给服务器处理, 通常是用户在表单中输入的信息。

HEAD: 请求一个对象的信息, 而不是对象本身, 实体为空。

PUT: 向URL指定的服务器的位置上传对象, 实体为此对象。

DELETE: 删除URL指定的服务器中的对象, 实体为空。



部分状态码与短语:

200 OK: 请求成功, 返回的对象在实体中。

301 Moved Permanently: 请求的对象已转移, 其新的URL在首部行的Location中给出。

400 Bad Request: 错误的请求

404 Not Found: 请求的对象不在该服务器上

505 HTTP Version Not Supported: 请求报文的HTTP版本服务器不支持

2.2.4 用户与服务器交互: Cookie

HTTP服务器是无状态的, 不保存客户信息; 但客户端可以用cookie方式保存客户信息, 允许Web应用跟踪、识别用户。许多Web站点都使用cookie, 每台客户机都有很多不同网站的cookie文件。客户机中建立一个针对某服务器的cookie文件并由浏览器管理, 用于记录用户信息, 服务器的数据库保存不同cookie文件的识别码, 在HTTP请求、响应报文中使用cookie首部行。

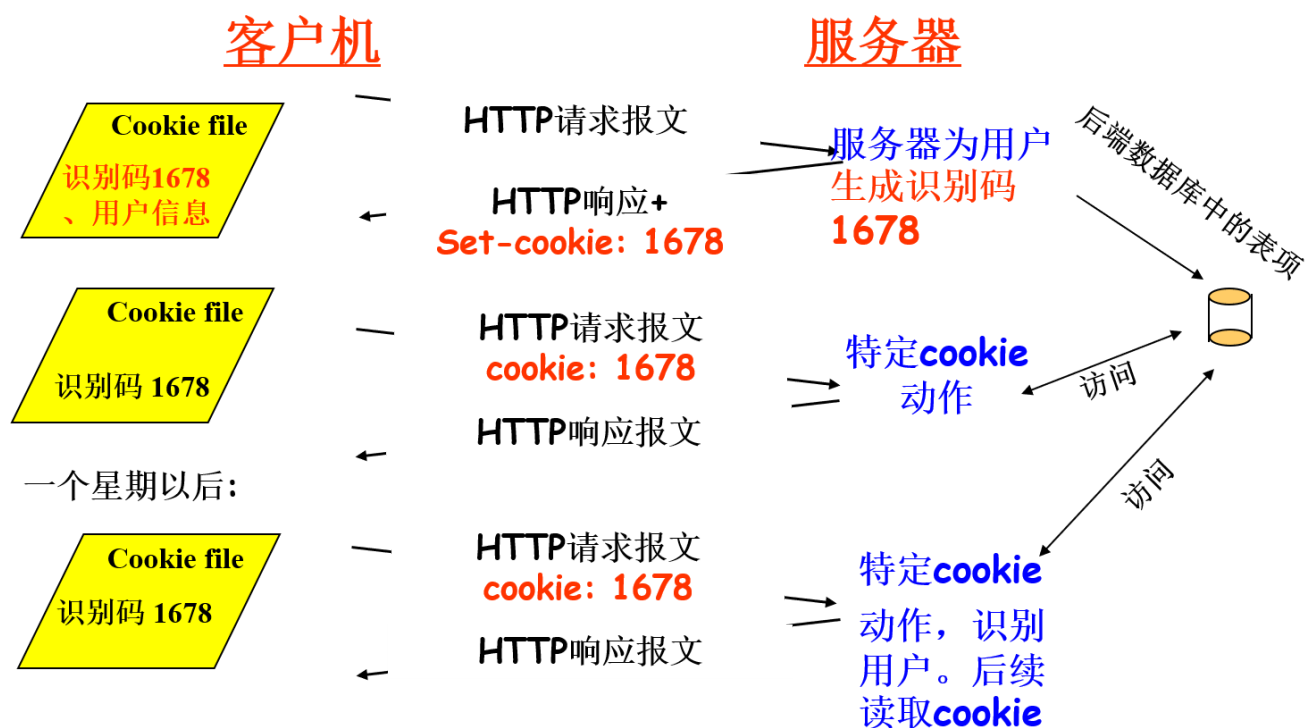


图2-4: cookie文件与识别码的关联过程

Cookie可以用于身份认证(存储用户名密码), 虚拟购物车(存储用户已选商品), 推荐广告(存储用户浏览习惯), 存储用户会话状态。缺点是Web站点可以收集用户信息, 不安全。

2.2.5 Web缓存

Web缓存器(Web cache), 也叫代理服务器(Proxy server), 位于客户机和服务器之间, 保存客户机最近访问过的对象的副本, 代表服务器响应客户机的HTTP请求。

用户配置浏览器将HTTP请求指向Web缓存器, 若对象在缓存中, 缓存器返回对象, 若不在, 缓存器向目标服务器发出请求, 接收对象后转发给客户机, 并保存。缓存器兼任服务器和客户机角色。

客户机与Web缓存器之间是高速链路, Web缓存器和服务器之间是低速链路, 所以Web缓存可以减少Web响应的时间, 减少因特网的使用流量, 降低费用。

2.2.6 条件GET方法

Web缓存器中存储的对象可能过时，所以使用条件GET方法检查并确保自己的对象拷贝为最新。Web缓存器第一次请求对象，目标服务器返回响应中包括对象的最后修改时间**Last-modified:date1**，此后Web缓存器定期向相应服务器发送条件GET报文，检查该对象是否已被修改，方法为GET，且包含首部行**If-modified-since:date1**，若对象未被修改，则目标服务器在状态行中响应：**304 Not Modified**，否则，在报文实体中返回修改后的对象。

2.3 文件传输协议：FTP

文件传输协议是应用层协议，基于运输层TCP，作用是本地主机向远程主机**上传或下载**文件，采用C/S结构，用户通过一个FTP**用户代理**与FTP交互。

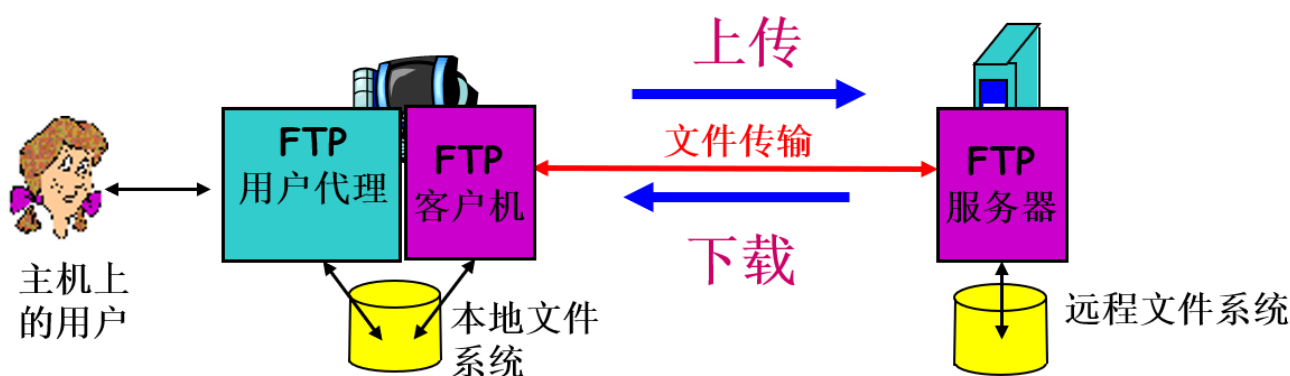


图2-5：文件传输协议

文件传输过程首先在FTP客户机进程与FTP服务器进程之间建立TCP连接，再输入用户名和口令进行身份验证，验证成功后进行文件传输。

FTP与HTTP都能基于TCP传输文件，但FTP使用了两个并行的TCP连接：**控制连接和数据连接**。

控制连接在两机控制进程间**传输控制信息**，如用户名、口令、操作命令与响应等，由**客户发起**，整个绘画过程中一直保持连接。服务器控制进程使用**TCP 21号端口**，客户机进程发送FTP命令，服务器返回响应。

都在控制连接上传，**ASCII 形式**

命令示例 (客户机发出):

- ✓ **USER *username***
- ✓ **PASS *password***
- ✓ **PORT *number*** 告知服务器
客户机所用端口号
- ✓ **LIST** 返回当前文件目录
- ✓ **RETR filename** 下载文件
- ✓ **STOR filename** 上传文件

响应示例 (服务器返回):

状态码 + 短语

- ✓ **331 Username OK,
password required**
- ✓ **125 data connection
already open; transfer
starting**
- ✓ **425 Can' t open data
connection**
- ✓ **452 Error writing file**

图2-6: FTP命令与响应

数据连接在两机数据进程间**传输文件**，服务器在控制连接上收到**文件传输的命令**，并收到PORT命令得知客户数据进程端口号。**服务器数据进程在TCP20端口主动发起建立数据连接**，在该连接上**传输文件**，完成后**关闭连接**。**每个文件的传送都要建立和关闭一次数据连接**。

控制连接是持续的，整个FTP会话期间一直保持；数据连接是非持续的，会话中每次文件传输都要打开和关闭数据连接。

FTP协议是有状态的，服务器在会话期间存储用户状态，但限制了FTP同时维持的会话数量；HTTP协议是无状态的，服务器不存储用户的会话状态，开销更小。

2.4 电子邮件应用

2.4.1 概述

电子邮件系统的主要部分为用户代理、邮件服务器、邮件传输协议。

用户代理(user agent): 允许用户发送、接收、撰写、阅读、回复、保存邮件。发邮件时，**用户代理向发送方邮件服务器发送邮件**，并存放在输出报文队列中；收邮件时，**用户代理从接收方邮件服务器的对应邮箱中获取邮件**。

邮件服务器(mail server): 邮件服务器由输出报文队列和用户邮箱组成。邮件传输过程分为三阶段，从发送方用户代理到发送方邮件服务器的输出报文队列，使用SMTP或HTTP；发送方邮件服务器到接收方邮件服务器，保存到接收用户邮箱中，使用SMTP；接收方用户代理访问接收方邮件服务器，从相应邮箱下载邮件，使用POP3或IMAP或HTTP。

简单邮件传送协议SMTP(Simple Mail Transfer Protocol): 作用为**从发送用户代理向发送方邮件服务器发送邮件**；**从发送方邮件服务器向接收方邮件服务器发送邮件**，是应用层协议，基于TCP，服务器进程端口为25。

alice@crepes.fr 向 bob@hamburger.edu 发送

服务器
准备好

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

发件方问候

准备发送

收件人

发送邮件

成功接收

关闭连接

图2-7：在邮件服务器间的SMTP交互

2.4.2 SMTP与HTTP对比

共同点：都传送文件；都使用持续连接，即一个TCP连接上可传送多个对象或电子邮件；单独都只能传送7位ASCII码格式的报文。

区别一：HTTP是拉协议，TCP连接由想获取文件的设备发起；SMTP是推协议，TCP连接是由要发送文件的设备发起。

区别二：对含有多个对象的文档，HTTP把每个对象分别封装在对应的HTTP响应报文中，SMTP则把所有对象放在一个报文中发送。

2.4.3 邮件报文格式

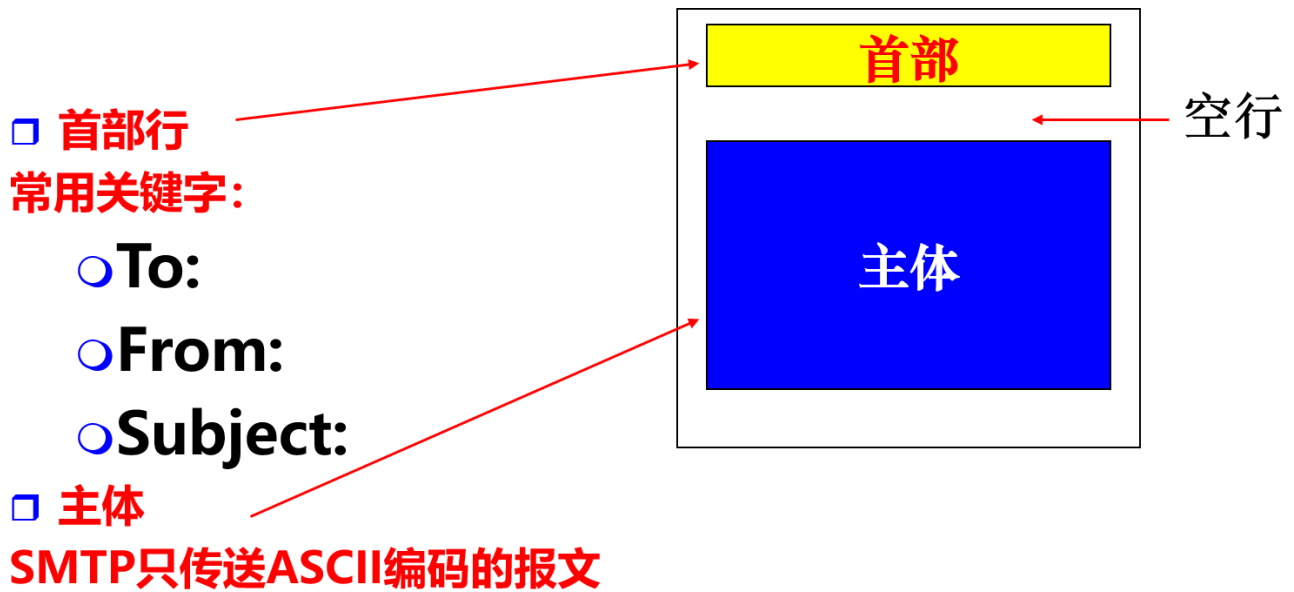


图2-8：邮件报文格式

2.4.4 邮件访问协议

邮件访问协议的作用是用户代理从邮件服务器上读取邮件，分三种：POP3、IMAP、HTTP。

POP3(Post Office Protocol 3)：第三版的邮局协议，简单但功能有限。首先由客户机进程发起并**建立TCP连接**，服务器进程使用**端口110**侦听。之后开始**读取邮件**，分三个阶段：特许：用户代理发送用户名和口令，**身份认证**；事务处理：用户代理取回邮件报文，可以**下载并保留**、**下载并删除**；更新：邮件服务器删除带有标记的报文，并结束会话。

1. 特许

□ 客户机命令:

○ user: 用户名

○ pass: 口令

□ 服务器响应

○ +OK

○ -ERR

2. 事务处理, 客户机命令:

□ list: 给出邮件列表

□ retr: 获取邮件

□ dele: 删除邮件

3. 更新, 客户机命令:

□ quit: 断开连接

```
S: +OK POP3 服务器 ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

```
C: list
S: 1 498
S: 2 912
```

列出邮件编号、大小

```
S: .
C: retr 1
S: <message 1 contents>
```

取邮件1, 并删除

```
S: .
C: dele 1
C: retr 2
S: <message 2 contents>
```

取邮件2, 并删除

```
S: .
C: dele 2
C: quit
S: +OK POP3 服务器 signing off
```

图2-9: POP3示例

IMAP(Internet Mail Access Protocol): 因特网邮件访问协议, 功能更强, 也是基于TCP, 是一个**联机协议**, 用户代理可以操作服务器的邮箱, 如建立文件夹、移动邮件等, 而POP3不行。可在用户代理上浏览邮件的摘要信息, 再决定是否下载整个邮件, 较复杂。

当用户代理为浏览器时, 使用HTTP协议, 邮件服务器之间传送仍使用SMTP。

2.5 目录服务DNS

因特网中标识主机的方式有主机名, 即有结构的字母数字如Einsturing.top, 或IP地址, 即有结构的4个或16个字节二进制, 如IPv4地址(点分十进制)。存在两种地址转换的问题。

DNS(Domain Name System): 域名系统/目录服务, **实现主机名与IP地址的相互转换**。由分布式数据库和DNS协议组成。分布式数据库由分层的DNS服务器实现, 记录了主机名与IP地址的对应信息; DNS协议用于客户机查询分布式数据库, 获取对应的主机名或IP地址。是应用层协议, 基于UDP, 采用**C/S**方式工作, 服务器进程使用端口UDP 53。

DNS服务器是如运行BIND软件的UNIX计算机, 使用环境在于**为其他应用层协议(如HTTP、SMTP、FTP), 提供主机名代IP地址的解析**。

例如用户浏览器访问网页, 会有以下过程: 用户机上启用DNS客户机进程; 浏览器将URL中**主机名**, 传给DNS客户机进程; DNS客户机向默认DNS服务器**发送包含主机名的DNS请求**; DNS服务器查数据库后(可能要进行多级服务器查询), 返回**含有对应IP地址**的应答; 浏览器使用该IP地址向HTTP服务器发起TCP连接, 然后传送HTTP请求。

DNS提供主机名与IP地址的相互转换服务, 主机别名到规范名的映射服务。

负载分配: 多个提供相同服务的服务器使用同一个名字。

2.6 P2P应用

端系统(对等方)之间直接通信, 很少或者不使用服务器。典型应用有P2P文件分发如BT下载, P2P数据库。

BT(BitTorrent)是一个P2P文件分发协议, 实现单个源向大量对等方快速分发文件。Torrent意为洪流, 指参与文件分发的所有对等方。

P2P文件分发的特点为: 洪流中, 对等方**彼此下载**等长度的文件块; 一个对等方可在任何时候加入洪流; **下载文件块的同时**, 也为洪流中其他对等方**上传**文件块; 对等方获得整个文件后可以离开, 也可以留下, 继续提供上传。

追踪器(tracker): 每个洪流有一个追踪器, 用于跟踪此洪流的对等方, 一个对等方加入洪流时, **在追踪器中注册**, 并周期性地通知洪流, 是否仍在洪流中。

BT工作原理: 一个对等方A加入洪流时, 追踪器从中**随机选择一些对等方**, 将其IP地址列表发给A; **A分别与列表中对等方建立TCP连接**, 从多个对等方获取文件块, 同时, 也通过追踪器为洪流中其他对等方提供已有文件块的下载; 下载完成后, **组装**为一个完整的文件。

2.7 套接字编程

网络应用程序通常包括客户机程序和服务器程序, 通过**套接字(socket)读写数据**实现网络通信, 开发前应先选择运输层协议, **双方进程通过套接字向网络发收报文**, 程序开发者可以控制应用层短, **不能控制运输层端(只能选择使用TCP或UDP, 设置少数参数)**。

2.7.1 TCP套接字编程

先建立TCP连接, 再传输数据。客户机进程是连接的发起方, 服务器应**先准备好, 设备已经运行并且已经打开一个套接字(欢迎套接字)****, 准备接受任何客户机发起的连接请求。

1) 建立TCP连接: 客户机**创建一个本地套接字**, 并给出服务器进程地址(IP地址+端口号); 当服务器侦听到连接请求时, **创建一个新套接字(连接套接字)**, 并经过“**三次握手**”与对方建立TCP连接。

2) 传送用户数据。

流: 流入或流出某**进程**的一串字符序列。

输入流: 针对进程而言, 来自某个**输入源的数据**, 如来自键盘或来自套接字(网络)。

输出流: 针对进程而言, 去往某个**输出源的数据**, 如去往显示器或去往套接字(网络)。

以下示例程序采用TCP通信, 客户机从键盘读一行字符发给服务器, 服务器将字符转换成大写发给客户机。

```
#TCPClient.py
from socket import *
serverName = '127.0.0.1'
serverPort = 12000
#套接字的创建 第一个参数指示IPv4, 第二个参数表明是TCP
clientSocket = socket(AF_INET, SOCK_STREAM)
#执行三次握手并在客户和服务器之间创建一条TCP连接
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server: ', modifiedSentence.decode())
#关闭套接字
```

```

clientSocket.close()

#TCPServer.py
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
#让服务器聆听来自客户的TCP连接请求，参数定义了请求连接的最大数(至少为1)
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket,addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()

```

2.7.2 UDP套接字编程

UDP是一种**无连接**的服务，**没有连接所需的握手阶段**，数据传递以数据报为单位进行，数据报中含目的进程的地址（**IP地址和端口号**），提供不可靠的传输服务。

通信进程之间没有初始握手，**不需要欢迎套接字**；**没有流**与套接字相关联；发送主机将信息**封装生成数据报**再发送；接收进程**解封收到的数据报**，获得信息。

以下示例程序同上节，但采用UDP通信。

```

#UDPClient.py
from socket import *
serverName = '127.0.0.1'
serverPort = 12000
#套接字的创建 第一个参数指示IPv4，第二个参数表明是UDP
clientSocket = socket(AF_INET,SOCK_DGRAM)
sentence = input('Input lowercase sentence:')
clientSocket.sendto(sentence.encode(),(serverName,serverPort))
modifiedSentence,serverAddress = clientSocket.recvfrom(2048)
print('From Server: ',modifiedSentence.decode())
#关闭套接字
clientSocket.close()

#UDPServer.py
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_DGRAM)
serverSocket.bind(('',serverPort))
print('The server is ready to receive')
while True:
    sentence,clientAddress = serverSocket.recvfrom(2048)
    modifiedSentence = sentence.decode().upper()
    serverSocket.sendto(modifiedSentence.encode(),clientAddress)

```